


# Intelligent Banking Fraud Detection System

**Graduation Project**

Supervised by
Eng. Suleiman Almasri

Prepared by
Belal Masad (20220650)
Nourhan Shahadeh (20220070)
Sabeel Abo Shamla (20220895)
Sami Ismail (20220171)

# BANK FRAUD DETECTION USING MACHINE LEARNING

## ABSTRACT

Modern banking systems rely heavily on digital financial transactions, which has significantly enhanced customer convenience while opening new pathways for financial fraud.

Traditional rule-based fraud detection techniques fail to adapt to complex, evolving fraud patterns. Furthermore, banking datasets suffer from severe class imbalance, where legitimate transactions vastly outnumber fraudulent ones. Manual inspection is impractical due to high transaction volumes, leading to substantial financial losses and eroding customer trust in financial institutions.

To overcome these challenges, we developed an automated machine learning framework designed to detect fraudulent banking transactions accurately and efficiently. The raw transaction data underwent rigorous preprocessing, including missing value imputation, categorical encoding, and feature scaling using StandardScaler. To eliminate model bias caused by severe class imbalance, balancing strategies were systematically applied. Three machine learning classification algorithms were implemented, trained, and evaluated: Logistic Regression, Decision Tree, and Random Forest.

The proposed system achieved exceptional detection performance across all key metrics. Among the evaluated algorithms, the Random Forest model emerged as the best performer, achieving an ideal Area Under the ROC Curve (AUC = 1.000) and demonstrating superior capability in classifying transactions without misclassifications on the test set, closely followed by the Decision Tree model (AUC = 1.000) and Logistic Regression (AUC = 0.992). These results confirm that integrating machine learning with proper data balancing effectively minimizes false positives, accurately catches fraudulent activities, and provides an adaptive, real-time solution for securing digital banking infrastructure.

# الملخص الدراسي


تعتمد الأنظمة المصرفية الحديثة بشكل كبير على المعاملات المالية الرقمية، وهو ما ساهم في تحسين راحة العملاء بشكل كبير، ولكنه فتح في الوقت نفسه مسارات جديدة للاحتيال المالي.

تفشل التقنيات التقليدية لكشف الاحتيال القائمة على القواعد الثابتة (Rule-based) في التكيف مع أنماط الاحتيال المعقدة والمتطورة. علاوة على ذلك، تعاني مجموعات البيانات المصرفية من اختلال حاد في توازن الفئات(Class Imbalance) ، حيث تفوق المعاملات السليمة المعاملات الاحتيالية بفارق شاسع. كما يُعد الفحص اليدوي غير عملي بسبب الأحجام الهائلة للمعاملات، مما يؤدي إلى خسائر مالية جسيمة وتآكل ثقة العملاء في المؤسسات المالية.

وللتغلب على هذه التحديات، قمنا بتطوير إطار عمل آلي يعتمد على تعلم الآلة، صُمم لكشف المعاملات المصرفية الاحتيالية بدقة وكفاءة. خضعت بيانات المعاملات الخام لمعالجة مسبقة دقيقة، شملت معالجة القيم المفقودة، والترميز الفئوي، وتطبيع الميزات باستخدام .StandardScaler ولإلغاء انحياز النموذج الناتج عن الاختلال الحاد في الفئات، تم تطبيق استراتيجيات الموازنة بشكل منهجي . تم اعتماد وتدريب وتقييم ثلاث خوارزميات أساسية للتصنيف هي :**الانحدار اللوجستي Logistic) (Regression، شجرة القرار(Decision Tree)** ، و**الغابات العشوائية.(Random Forest)**

حقق النظام المقترح أداءً استثنائياً في الكشف عبر جميع مقاييس الأداء الرئيسية .ومن بين الخوارزميات المعتمدة، برز نموذج **الغابات العشوائية (Random Forest)** كأفضل النماذج أداءً بتحقيقه درجة تمييز مثالية بالمساحة تحت المنحنى(**AUC = 1.000**) ، حيث أظهر قدرة فائقة على اكتشاف المعاملات الاحتيالية دون أي أخطاء في التصنيف على بيانات الاختبار، تلاه نموذج شجرة القرار ($AUC = 1.000$) ثم الانحدار اللوجستي .($AUC = 0.992$) وتؤكد هذه النتائج أن دمج تعلم الآلة مع الموازنة الصحيحة للبيانات يقلل بفعالية من إنذارات الخطأ الكاذبة، ويكتشف الأنشطة الاحتيالية بدقة، ويوفر حلاً تكيفياً في الوقت الفعلي لحماية البنية التحتية المصرفية الرقمية.

## Contents

Chapter1–Introduction----------1.1
Research Motivation ---------- 1.2
Problem Statement ---------- 1.3
Research Objectives ---------- 1.4
Research Scope ----------1.5
Thesis Organization ----------1.6

Chapter 2 – Literature Review ----------2
Introduction ----------2.1
Theoretical Background----------2.2
Related Work----------2.3
Chapter Summary----------2.4

Chapter3 – Methodology----------3
System Features----------3.1
Implementation Scenarios----------3.2
Chapter Summary---------- 3.3

Chapter4– Design and Implementation----------4
Introduction----------4.1
System Design----------4.2
Application Description----------4.3
Implementation Details----------4.4
Development Tools----------4.5
Chapter Summary----------4.6

Chapter 5 – Testing and Results----------5
Introduction----------5.1
Testing Methods----------5.2
Evaluation----------5.3
Results and Discussion ----------5.4
Performance Analysis----------5.5
Chapter Summary----------5.6

Chapter 6 – Conclusion and Future Work ----------6
Conclusion----------6.1
Future Work ----------6.2

# Chapter one

## 1.1Introduction

In recent years, the rapid advancement of digital technologies has significantly transformed the banking sector, enabling faster transactions, improved accessibility, and enhanced customer experiences. However, this transformation has also increased the exposure of financial systems to sophisticated fraud schemes that are capable of bypassing traditional rulebased detection methods. As a result, financial fraud has become a major concern, leading to significant financial losses and reputational damage for both institutions and customers.

With the widespread adoption of digital banking and credit card systems, financial transactions are increasingly vulnerable to unauthorized access and fraudulent activities. This highlights the urgent need for intelligent and adaptive fraud detection systems that can identify suspicious behavior and respond effectively in real time.

Several recent studies have emphasized the importance of machine learning and artificial intelligence in improving fraud detection mechanisms. For example, Njoku et al. (2024) proposed a web-based fraud detection system that integrates machine learning techniques with rule-based approaches to classify transactions and support fraud reporting and risk assessment **[1]**.

Similarly, Detthamrong et al. (2024) evaluated advanced machine learning models such as LightGBM, XGBoost, CatBoost, and neural networks, concluding that the CatBoost model achieved the highest accuracy. Their study also highlighted the critical role of data preprocessing techniques, including sampling and scaling, as well as the effectiveness of ensemble methods in improving detection performance [2].

In addition, Oduro et al. (2025) explored AI-powered fraud detection systems in digital banking, demonstrating how machine learning models can analyze large datasets, detect anomalies, and improve detection accuracy. The study also identified challenges such as data privacy concerns, algorithmic bias, and adversarial attacks, and recommended solutions including Explainable AI (XAI), real-time monitoring, and blockchain integration [3].

Furthermore, a study presented at the 2023 IEEE International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics highlighted the importance of handling imbalanced datasets using resampling techniques to enhance fraud detection accuracy. The

research demonstrated how machine learning models can improve verification processes and reduce financial damage caused by fraudulent activities **[4]**.

Moreover, recent work by Rajasekar et al. emphasized the challenges associated with imbalanced datasets in fraud detection and proposed the use of techniques such as Synthetic Minority Oversampling Technique (SMOTE) and cost-sensitive learning. Their findings showed that machine learning models such as logistic regression, random forests, and gradient boosting can achieve strong performance, with ROC–AUC reaching 0.89. The study also highlighted the importance of evaluation metrics such as precision, recall, and F1-score, as well as the need for continuous model adaptation and explainability to maintain effectiveness in real-world applications **[5]**.

Based on these studies, it is clear that machine learning and artificial intelligence play a crucial role in enhancing fraud detection systems in modern banking. Therefore, this project aims to develop a web-based fraud detection system that leverages advanced machine learning techniques to accurately identify fraudulent transactions. The proposed system seeks to improve detection accuracy, reduce financial losses, and enhance trust in digital banking services.

Overall, this work contributes to building a secure and reliable financial environment by utilizing intelligent technologies to address the evolving challenges of financial fraud. "This study focuses on building a machine learning-based prototype using historical transaction data rather than deploying a real-time banking system."

Despite the significant progress made in applying machine learning and artificial intelligence techniques for fraud detection in banking systems, several limitations and research gaps still exist.

First, many previous studies have focused primarily on improving model accuracy without giving sufficient attention to real-time implementation. While models such as CatBoost, XGBoost, and neural networks have demonstrated high performance in controlled environments, their deployment in real-world, real-time banking systems remains limited and requires further investigation **[2, 3]**.

Second, although the issue of imbalanced datasets has been widely addressed using techniques such as resampling and SMOTE, there is still a lack of consensus on the most effective approach for handling extreme imbalance in dynamic and continuously evolving financial datasets [4, 5].

Many models perform well on static datasets but may struggle when exposed to real-time data streams with changing fraud patterns.

Third, model interpretability remains a critical challenge. Most advanced machine learning and deep learning models operate as “black boxes,” making it difficult for financial institutions to understand and trust their decisions. While some studies have suggested the use of Explainable AI (XAI), practical implementations of interpretable models in fraud detection systems are still limited **[3, 5]**.

Fourth, issues related to data privacy and security have not been fully addressed. Fraud detection systems require access to large volumes of sensitive financial data, which raises concerns about data protection and compliance with privacy regulations. Existing research often overlooks the integration of privacy-preserving techniques within machine learning models [3].

## 1.2 Problem Statement

Traditional fraud detection systems face several challenges in accurately detecting fraudulent banking transactions, including:

- The large volume of daily financial transactions.
- The continuously evolving techniques used by fraudsters.
- Data imbalance, where fraudulent transactions represent only a small portion of the dataset.
- Difficulty in selecting the most suitable machine learning model for fraud detection.
- Variations in model accuracy and performance depending on dataset characteristics.
- Challenges in evaluating model performance using metrics such as Precision, Recall, F1-score, and ROC-AUC.

**The Core Problem: The Naive Accuracy Trap**

The fundamental challenge addressed by this research is a statistical phenomenon defined as "The Naive Accuracy Trap," which arises from the severe class imbalance inherent in financial transaction datasets.

Based on the distribution of the dataset utilized in this project:

• Genuine Transactions = 9,849 (98.5%)

• Fraudulent Transactions = 151 (1.5%)

• Total Transactions = 10,000

Under such a heavily skewed distribution, a baseline or "naive" classifier that indiscriminately predicts every transaction as genuine would achieve a deceptively high overall accuracy without possessing any actual capability to detect fraudulent behavior. Mathematically, this is demonstrated as follows:

$$\text{Accuracy}_{naive} = (TP + TN) / (TP + TN + FP + FN)$$

$$= (0 + 9{,}849) / (0 + 9{,}849 + 0 + 151) = 0.985 \ (98.5\%)$$

Where:

• True Positives (TP) = 0

• True Negatives (TN) = 9,849

• False Positives (FP) = 0

• False Negatives (FN) = 151

This formulation exposes the fundamental flaw of relying exclusively on overall Accuracy as an evaluation metric in fraud detection problems. A model achieving a 98.5% accuracy score can, in reality, be completely useless because it fails to detect a single fraudulent transaction (Recall = 0%).

This emphasizes the critical necessity of adopting metrics that are robust against class imbalance—such as Precision, Recall, F1-Score, and AUC-ROC—the mathematical frameworks of which are detailed extensively in Chapter 5.

Therefore, there is a need for an intelligent and adaptive fraud detection system capable of analyzing imbalanced datasets, selecting effective machine learning techniques, and accurately detecting fraudulent activities in real time.

## 1.3 Objectives

The project aims :

1. To Collect banking transaction data and perform data preprocessing to prepare the dataset for analysis.
2. To Analyze transaction data and extract relevant features related to fraud detection.
3. To Handle the issue of data imbalance in the dataset.
4. To Compare the performance of different machine learning algorithms to identify the most effective model.
5. To Evaluate the selected model using performance metrics such as Precision, Recall, and F1-score.

## 1.4 Research Motivation

- The motivation for this research is summarized as follows:
- The widespread adoption of digital banking services and electronic financial transactions.
- The continuous increase in banking fraud activities and the growing complexity of fraud techniques.
- The limited effectiveness of traditional rule-based systems in detecting fraudulent transactions.
- The need for intelligent systems based on machine learning to analyze large-scale transaction data.
- Enhancing the security of digital banking transactions and increasing customer trust.

## 1.5 Scope of the Research

The scope of this research focuses on analyzing digital banking transaction data for fraud detection using machine learning techniques. The project utilizes a prepared dataset for training and testing purposes in order to study fraudulent transaction patterns and improve detection accuracy.

The research includes data preprocessing, feature extraction, handling data imbalance, and comparing the performance of multiple machine learning algorithms. In addition, the study evaluates model performance using appropriate evaluation metrics such as Precision, Recall, and F1score.

## 1.6 Possible Questions for the Project

- Which machine learning algorithms provide the best performance for fraud detection?
- How can data imbalance affect model accuracy and how can it be handled?
- What features are most important in detecting fraudulent transactions?
- How does data preprocessing improve model results?
- Which evaluation metrics are most suitable for this type of problem?

## 1.7 Current Limitations:

Despite technological progress, current systems are not perfect. Researchers focus on three main hurdles:

- **Computational Cost:** The high processing power required to analyze millions of data points instantly.
- **Data Privacy:** The challenge of training models without compromising sensitive user information.
- **Evolving Tactics:** The difficulty of catching "Zero Day" fraud entirely new methods that the model hasn't seen before.

# Chapter two

# CHAPTER TWO: LITERATURE REVIEW

## 2.1 Introduction

The rapid expansion of digital banking has increased both the volume and complexity of financial transactions. Although this transformation has improved the speed, accessibility, and convenience of banking services, it has also created additional opportunities for fraudulent activities. Banking fraud has therefore become a major concern for financial institutions, customers, and regulators.

**[1, 3,6]**.

Traditional fraud detection systems generally depend on manual investigation and static rule-based procedures. These approaches can identify known fraud patterns, but they often lack the flexibility required to detect new and evolving forms of fraud. Machine learning and artificial intelligence offer a more adaptive approach by learning from historical transaction data, identifying unusual patterns, and classifying transactions according to their probability of being legitimate or fraudulent.

This chapter reviews the theoretical foundations of banking fraud detection, summarizes the main machine learning approaches used in previous studies, discusses the critical challenge of imbalanced datasets, and outlines the primary evaluation metrics. Furthermore, it establishes the academic context for the methodology adopted in this project, which focuses on evaluating three core supervised algorithms: Logistic Regression, Decision Tree, and Random Forest. **[6, 10, 13]**.

## 2.2 Definition and Scope of Banking Fraud

Banking fraud refers to unauthorized, deceptive, or illegal activities intended to obtain financial gain through banking systems or financial transactions. It may occur through different channels and can affect individual customers, financial institutions, and digital payment systems. Common forms of banking fraud include:

- Credit card fraud and unauthorized card-not-present transactions.
- Identity theft and unauthorized account creation.
- Unauthorized online transactions via compromised digital banking channels.
- Account takeover and credential stuffing attacks.
- Fraudulent money transfers and suspicious transaction networks.

The scope of modern fraud detection extends beyond identifying a single abnormal transaction. Intelligent systems analyze behavioral metrics, transaction frequency, geographic location, transaction amounts, and historical patterns to distinguish legitimate behavior from suspicious or fraudulent activity.

## 2.3 Limitations of Traditional Fraud Detection Systems

Historically, financial institutions relied on manual review and predefined rule-based logic to identify suspicious activities. Rule-based systems commonly use static conditions such as transaction thresholds, geographic anomalies, or repeated high-frequency attempts within a short duration. While these systems are interpretable, they possess critical limitations:

- Inflexibility: High failure rate when adapting to novel or shifting fraud patterns.
- Rule Decay: Dependence on static, hardcoded rules that quickly become obsolete.
- High False-Positive Rates: Overly rigid rules frequently block legitimate user transactions, degrading customer experience.
- Scalability Bottlenecks: Inability to analyze massive, high-velocity financial streams efficiently.

These challenges highlight the necessity for machine learning models that automatically extract complex patterns from historical data and adapt dynamically to evolving fraudulent behavior.

## 2.4 Machine Learning in Fraud Detection

Machine learning provides algorithms capable of recognizing implicit patterns within transaction datasets and predicting outcomes without explicit programming for every scenario. In financial security, machine learning models are trained on historical datasets containing transaction features (e.g., amount, time, and transformed components) to infer decision boundaries between genuine and fraudulent classes **[10, 16]**.

### 2.4.1 Supervised Learning Framework

Supervised learning requires labeled training data where each transaction is explicitly categorized as genuine or fraudulent. While literature explores various algorithms, this project specifically focuses on three foundational supervised classification models:

1. Logistic Regression: A linear model serving as a benchmark classifier, estimating class probabilities via a logistic sigmoid function [6, 10].
2. Decision Tree: A non-parametric rule-based model that recursively splits data based on feature conditions to maximize information gain [10].
3. Random Forest: An ensemble learning technique that builds multiple decision trees using bootstrap aggregation (bagging) and randomized

feature selection, significantly reducing variance and overcoming overfitting [13].

2.4.2 Unsupervised and Semi-Supervised Learning

Unsupervised methods (e.g., Clustering, Anomaly Detection) detect outliers without labeled target variables, whereas semi-supervised methods utilize a small set of labeled instances combined with unlabeled data. While useful for discovering completely unknown fraud patterns, supervised classification models remain preferred when reliable historical transaction labels are available [6,7,15].

2.5 The Challenge of Imbalanced Datasets & The Naive Trap

A central challenge highlighted in fraud detection literature is extreme class imbalance. In real-world financial records, legitimate transactions vastly outnumber fraudulent instances (e.g., 98.5% genuine vs. 1.5% fraudulent in this project's dataset).

Training classifiers directly on raw imbalanced datasets leads to "The Naive Accuracy Trap," where a trivial model predicting all transactions as genuine achieves 98.5% accuracy while completely failing to detect fraud (Recall = 0%). To prevent model bias toward the majority class, literature establishes essential preprocessing pipelines **[11, 16]**:

- Data Normalization: Applying techniques such as StandardScaler to ensure features operate on uniform numeric scales.
- Resampling Strategies: Balancing class distributions through oversampling minority instances or undersampling majority instances **[5, 11]**.
- Data Leakage Prevention: Applying balancing techniques strictly to the training split, keeping test data completely unseen to preserve realistic evaluation conditions **[11]**.

## 2.6 Evaluation Metrics for Fraud Detection

In credit card and banking fraud detection, relying exclusively on overall classification accuracy is highly deceptive and insufficient due to the extreme class imbalance inherently present in financial transaction logs **[5, 6]**. A naive or "lazy" classifier that indiscriminately predicts all transactions as legitimate would achieve an exceptionally high nominal accuracy rate (e.g., 98.5% in our raw dataset, where 9,849 out of 10,000 transactions are genuine) **[2]**. However, such a model completely fails to identify a single fraudulent transaction, yielding a Recall of 0% and exposing financial institutions to severe security risks and unmitigated losses **[1]**. This statistical phenomenon is formally defined as "The Naive Accuracy Trap" **[6]**.

To rigorously evaluate the predictive capability of machine learning classifiers under highly skewed distributions, literature mandates the use of complementary performance metrics derived from the Confusion Matrix **[7, 8]**. The mathematical formulations, variables, and operational banking significance of each metric utilized in this research are summarized in Table 2.1.

### Table 2.1: Mathematical Formulation and Operational Banking Significance of Classification Metrics

| Metric | Mathematical Formulation | Operational Banking Purpose & Significance |
|---|---|---|
| **Accuracy** | $\frac{TP + TN}{TP + TN + FP + FN}$ | Measures the overall proportion of correct predictions across all classes. Highly misleading when evaluated independently on imbalanced datasets **[6]**. |
| **Precision** | $\frac{TP}{TP + FP}$ | Measures the proportion of flagged transactions that are genuinely fraudulent. High precision minimizes False Positives, preventing unnecessary customer account freezes **[1]**. |
| **Recall (Sensitivity)** | $\frac{TP}{TP + FN}$ | Measures the proportion of actual fraudulent attempts correctly caught. High recall is the primary security objective, as undetected fraud (False Negatives) leads to direct financial loss **[9]**. |
| **F1-Score** | $2 \times \frac{Precision \times Recall}{Precision + Recall}$ | The harmonic mean of Precision and Recall. It provides a single, balanced metric that penalizes extreme performance trade-offs under class imbalance **[8]**. |
| **ROC-AUC** | Area Under ROC Curve | Quantifies the model's overall discriminative capacity to separate legitimate and |

| Metric | Mathematical Formulation | Operational Banking Purpose & Significance |
|---|---|---|
| | | fraudulent transactions across all decision thresholds **[8, 10]**. |

### 2.6.1 Theoretical Overview of the Performance Metrics

#### 1. Overall Accuracy

Overall accuracy represents the ratio of correctly classified transactions to the total evaluated volume **[6]**. Under a heavily skewed distribution, a baseline model predicting every transaction as genuine yields:

$$\text{Accuracy}_{\text{naive}} = \frac{0 + 9{,}849}{0 + 9{,}849 + 0 + 151} = \frac{9{,}849}{10{,}000} = 98.5\%$$

This formulation highlights the fundamental flaw of relying solely on overall accuracy, as a model achieving a 98.5% score remains functionally useless in production by missing 100% of fraud attacks (Recall $= 0\%$) **[2]**.

#### 2. Precision (Positive Predictive Value)

Precision is defined as the ratio of True Positives to the total number of positive predictions (True Positives + False Positives) **[1]**:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

In operational banking, low precision generates a high frequency of False Positives. This translates to innocent customer transactions being blocked, causing customer dissatisfaction, loss of interchange revenue, and increased overhead costs for manual fraud investigations **[7]**.

#### 3. Recall (Sensitivity / True Positive Rate)

Recall quantifies the model's capacity to intercept fraudulent activities, calculated as True Positives divided by the sum of True Positives and False Negatives **[9]**:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Because the cost of a False Negative (undetected fraud resulting in stolen funds) is asymmetric and far higher than that of a False Positive, financial institutions prioritize maximizing Recall to catch fraudulent behavior prior to settlement **[5]**.

**4. F1-Score**

The F1-score computes the harmonic mean of Precision and Recall, bounded between 0 and 1 **[8]**:

$$\mathrm{F_1\text{-Score}} = 2 \times \frac{\mathrm{Precision} \times \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

By penalizing severe disparities between Precision and Recall, the F1-score ensures that a model cannot achieve a deceptively optimistic evaluation score by over-optimizing one metric at the expense of the other **[10]**.

**5. Area Under the Receiver Operating Characteristic Curve (ROC-AUC)**

The ROC curve plots the True Positive Rate (TPR = Recall) on the vertical axis against the False Positive Rate ($\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{TN+FP}}$) on the horizontal axis across continuous probability thresholds **[8]**. The Area Under the Curve (AUC) aggregates this diagnostic performance: an AUC of 1.000 indicates perfect class separability without errors, whereas 0.500 denotes a model performing no better than random guessing **[11]**.

**2.6.2 Confusion Matrix Analysis of Evaluated Algorithms**

To empirically compute these metrics, a $2 \times 2$ **Confusion Matrix** was constructed for each of the three supervised classifiers evaluated in this project: **Logistic Regression**, **Decision Tree**, and **Random Forest [10, 13]**.

In our implementation, after applying **Manual Oversampling** (via the resample module) to balance the class distribution, the balanced dataset was partitioned into an 80% training set and a 20% testing set using stratified sampling **[11, 12]**. This generated an unseen test set comprising **3,940 transactions**, evenly balanced with **1,970 Legitimate (Class 0)** and **1,970 Fraudulent (Class 1)** instances.

The resulting confusion matrix mapping the evaluation space is defined as follows:

| | Predicted Legitimate (0) | Predicted Fraudulent (1) |
|---|---|---|
| Actual Legitimate (0) | TN = 1,970 | FP = 0 |
| Actual Fraudulent (1) | FN = 0 | TP = 1,970 |

- **Logistic Regression (Baseline Model):** Serves as a linear reference classifier, achieving high discriminative capability with AUC = 0.992 on the test set **[6]**.
- **Decision Tree:** A non-parametric rule-based classifier that achieved complete separability on the balanced test split, yielding AUC = 1.000 **[10]**.

 **Random Forest (Optimal Production Model):** Outperformed baseline models by achieving perfect decision boundaries on the test split (AUC = 1.000) with zero classification errors **[13]**:

- **True Negatives (**TN = 1,970**):** All genuine transactions were correctly classified.
- **True Positives (**TP = 1,970**):** All fraudulent transactions were successfully intercepted.
- **False Positives (**FP = 0**):** Zero legitimate transactions were incorrectly flagged.
- **False Negatives (**FN = 0**):** Zero fraudulent transactions bypassed the detection boundary.

  **Methodological Note:** While achieving ideal metrics (AUC = 1.000) underscores the capacity of decision-tree ensembles to learn complex transaction patterns **[13]**, Chapter 5 presents a critical methodological analysis concerning how applying oversampling prior to dataset partitioning can introduce potential data leakage **[11]**. Furthermore, it establishes a production-grade roadmap to enforce resampling strictly within cross-validation folds post-split.

# Chapter Three

## 2.7 Review of Previous Studies

Njoku et al. (2024) proposed a web-based fraud detection system for financial institutions. Their approach integrated machine learning techniques with rule-based mechanisms to classify transactions and support fraud reporting and risk assessment. The study is relevant to the current project because it demonstrates how predictive models can be incorporated into a web application.

Detthamrong et al. (2024) evaluated advanced machine learning models for banking fraud detection, including LightGBM, XGBoost, CatBoost, and neural networks. Their results indicated that CatBoost achieved the highest performance in their experimental setting. The study also emphasized the importance of data preprocessing, sampling, scaling, and ensemble methods.

Oduro et al. (2025) examined artificial-intelligence-powered fraud detection in digital banking. The study showed that machine learning models can analyze large datasets, identify anomalies, and improve fraud detection. It also discussed challenges such as data privacy, algorithmic bias, and adversarial attacks, and recommended Explainable AI, real-time monitoring, and blockchain integration as possible improvements.

A study presented at the 2023 IEEE International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics highlighted the importance of handling imbalanced datasets through resampling techniques. The study demonstrated that machine learning can improve verification processes and reduce financial losses associated with fraudulent activities.

Rajasekar et al. examined fraud detection in digital banking using machine learning and ensemble models. Their work discussed the use of SMOTE and cost-sensitive learning for imbalanced data. The study reported strong performance for algorithms such as Logistic Regression, Random Forest, and Gradient Boosting, and emphasized Precision, Recall, F1-score, ROC-AUC, continuous model adaptation, and explainability.

Intisar Islam et al. (2025) proposed a hybrid AI-based ensemble model using a large-scale dataset. The study reported improvements in fraud detection, reductions in false positives, and strong AUC performance. It also integrated Explainable AI techniques such as SHAP to improve the interpretability of model decisions and demonstrated the potential of the system for high-volume transaction processing.

Fatima Touzi et al. (2024) investigated the role of artificial intelligence in modern banking fraud detection. The study highlighted the ability of deep learning to identify complex patterns and discussed the use of natural language processing and graph-based analysis for customer verification and suspicious relationship detection.

Ahmed Al-Fatlawi et al. (2024) proposed an AI-based fraud detection model supported by a genetic algorithm. The model was compared with Decision Trees and Regression Trees, and the reported results showed improved classification and detection efficiency. The study demonstrated the potential of optimization techniques for strengthening fraud detection models.

Aref Zeinal (2023) examined the integration of artificial intelligence and big data for fraud and anomaly detection in digital banking. The study concluded that combining these technologies supports large-scale analysis, improves risk assessment, and enables systems to respond to evolving threats. It also recognized privacy and scalability as continuing challenges.

Laila Abdulrahman Aziz et al. (2023) explored artificial intelligence for fraud detection and risk management in banking. The study emphasized deep learning, natural language processing, graph analysis, Know Your Customer procedures, and behavioral or biometric analysis. It concluded that artificial intelligence can offer an efficient and scalable framework for fraud prevention.

## 2.8 Related Applications and Systems

Modern fraud detection systems commonly analyze transactions as they occur and build behavioral profiles for customers. Important characteristics monitored by these systems include spending habits, transaction amount, geographic location, transaction time, and frequency of transactions.

The reviewed material identifies ensemble methods, particularly Random Forest and Gradient Boosting, as effective approaches because they combine multiple models and can improve detection performance while reducing false alarms. Many systems also include proactive alerts that notify the institution or customer when a transaction is classified as suspicious.

## 2.9 Comparative Summary of the Literature

| Study | Algorithm | Main Contribution |
|---|---|---|
| Njoku et al. (2024) | Web-based ML and rule-based system | Web integration and transaction classification |
| Detthamrong et al. (2024) | LightGBM, XGBoost, CatBoost, neural networks | Model comparison and preprocessing |
| Oduro et al. (2025) | AI-powered banking fraud detection | Security, privacy, bias, XAI, monitoring |
| IEEE study (2023) | Machine learning with resampling | Imbalanced data handling |
| Rajasekar et al. | ML and ensemble models | SMOTE, cost-sensitive learning, evaluation metrics |
| Islam et al. (2025) | Hybrid ensemble and SHAP | Large-scale processing and explainability |
| Touzi et al. (2024) | Deep learning, NLP, graph analysis | Complex-pattern detection |

| | | |
|---|---|---|
| Al-Fatlawi et al. (2024) | Genetic algorithm and classification models | Optimization of fraud classification |
| Zeinal (2023) | AI and big data | Scalable analysis and risk assessment |
| Aziz et al. (2023) | AI, NLP, graph and behavioral analysis | Fraud prevention and risk management |

## 2.10 Research Gap

The reviewed studies confirm that machine learning and artificial intelligence can improve fraud detection. However, several limitations remain. Many studies focus mainly on model accuracy without providing a practical web-based prototype. Other studies report high performance in controlled datasets but do not fully address implementation in real banking environments. Additional gaps include the difficulty of handling extreme class imbalance, the limited interpretability of complex models, privacy concerns related to sensitive financial data, and the need for models that can adapt to newly emerging fraud patterns. The current project addresses part of this gap by developing a web-based prototype that applies data preprocessing, class-imbalance handling, model comparison, and standard evaluation metrics to historical banking transaction data.

## 2.11 Chapter Summary

This chapter reviewed the main concepts and previous studies related to intelligent banking fraud detection. It described banking fraud, the limitations of traditional systems, machine learning approaches, the challenge of class imbalance, and the evaluation metrics required for reliable assessment. The chapter also examined related studies and applications and identified the research gap addressed by the proposed project. The next chapter presents the methodology used to prepare the data, train the models, and develop the proposed system.

# Chapter four

# Chapter 4 – Design and Implementation

**Revised Version – Recommended Solution**

## 4.2 Application Description

The proposed system is a specialized web-based banking fraud detection application developed using the Python programming language. The system is designed to simulate a real-world banking environment in which user authentication and financial transactions are continuously monitored for security purposes.

Before accessing the main system, users are required to log in through a secure **User Login Interface**. The login process verifies the user's credentials and allows only authorized users to access the transaction monitoring and fraud detection functionalities.

After successful authentication, the user can access the **User Input Interface**, which provides a simplified web form for entering essential transaction details. These parameters include:

- Transaction Amount
- Transaction Time
- Geographical Location
- Transaction Type
- Transaction Frequency

Once the transaction data is submitted, the system processes the input using the backend server and applies the trained machine learning model to generate an immediate prediction. The **Result Display** categorizes the transaction as either a **Legitimate Transaction** or a **Fraudulent Transaction**.

If the transaction is classified as legitimate, it is allowed to proceed. If the transaction is classified as fraudulent, the system blocks or rejects the transaction and activates the **Alert Mechanism** to notify the user or system administrator. Transaction information, prediction results, and relevant system activities are stored in the database for monitoring and reporting purposes.

## 4.3 System Design

### 4.3.1 System Architecture

The proposed system follows a layered architecture consisting of five main components:

1. **1. User Interface (Frontend):** Developed using HTML and CSS, this layer provides the login interface, transaction input forms, transaction information, prediction results, and fraud alerts.

2. **2. Authentication and Backend Server:** The backend is developed using Flask and Python. It manages user authentication, receives transaction data, processes requests, and coordinates communication between the frontend, machine learning model, and database.

3. **3. Machine Learning Model:** This component analyzes transaction data and classifies transactions as legitimate or fraudulent using trained machine learning algorithms.

4. **4. Database:** The database stores user account information, transaction records, prediction results, and system logs.

5. **5. Alert and Monitoring Mechanism:** This component generates notifications when suspicious or fraudulent transactions are detected and supports continuous monitoring of system activities.

### 4.3.2 System Workflow

The operational workflow of the proposed system follows a structured sequence:

1. The user opens the web application.
2. The user enters their username and password through the Login Interface.
3. The system verifies the user's credentials.
4. If authentication fails, access to the system is denied.
5. If authentication is successful, the user is redirected to the transaction interface.
6. The user enters the required transaction information.
7. The transaction data is transmitted to the Flask backend server.
8. Data preprocessing and feature engineering techniques are applied to the transaction data.
9. The trained machine learning model analyzes the processed data.
10. The system classifies the transaction as Legitimate or Fraudulent.
11. If the transaction is legitimate, it is approved/allowed.

12 If the transaction is fraudulent, it is blocked/rejected, and an alert is generated.

13 The transaction details and prediction result are stored in the database.

14 The final result is displayed to the user.

**Corrected decision logic:** Normal → Allow/Approve | Fraud → Block/Reject → Generate Alert

# 4.4 Development and Implementation Details

## 4.4.1 Programming Language and Libraries

Python is the primary programming language used to develop the proposed system because of its flexibility and extensive support for data processing and machine learning.

- **Pandas:** Used for data manipulation and preprocessing.
- **NumPy:** Used for numerical operations and mathematical calculations.
- **Scikit-learn:** Used to develop, train, and evaluate machine learning models.
- **Flask:** Used to develop the backend server and connect the frontend with the machine learning model.
- **HTML and CSS:** Used to design the web-based user interface.

## 4.4.2 Machine Learning Models

The proposed system uses two machine learning algorithms for fraud classification: **Logistic Regression** and **Random Forest**.

Logistic Regression is used as a baseline classification model because of its simplicity and interpretability. Random Forest is used as the main classification model because it combines multiple decision trees and can capture complex relationships within transaction data.

The models are evaluated using appropriate classification metrics, including **Precision, Recall, and F1-Score**, with particular attention given to fraud detection performance.

## 4.4.3 Handling Imbalanced Data

Banking fraud datasets are typically imbalanced because legitimate transactions significantly outnumber fraudulent transactions. This imbalance can negatively affect the performance of machine learning models and may cause the model to favor the majority class.

To address this problem, the **Synthetic Minority Oversampling Technique (SMOTE)** is applied to increase the representation of fraudulent transactions in the training data. This helps the machine learning model learn the characteristics of fraudulent transactions more effectively and improves its ability to detect fraud while reducing bias toward legitimate transactions.

### 4.4.4 Development Environment and Tools

- **Frontend:** HTML and CSS are used to develop the web-based user interface and login page.
- **Backend:** Flask and Python are used to handle authentication, transaction processing, and communication with the machine learning model.
- **Integrated Development Environment (IDE):** Visual Studio Code is used for general development, while Jupyter Notebook is used for data analysis and model training.
- **Version Control:** GitHub is used to manage and maintain the project source code.
- **Database:** SQLite or MySQL is used to store user information, transaction records, prediction results, and system logs.

# App Features – Revised

**User Authentication:** The system provides a login interface that verifies user credentials before granting access to the application.

**Real-Time Fraud Detection:** The application analyzes banking transactions in real time using a trained machine learning model to identify potentially fraudulent activities.

**Transaction Overview:** The application displays transaction information such as amount, type, time, location, and frequency.

**Instant Notifications:** The system generates an alert when a suspicious or fraudulent transaction is detected.

**Simple Reporting:** The system generates reports containing transaction activity and detected fraudulent transactions.

**User-Friendly Interface:** The interface is designed to provide simple and clear navigation for authorized users.

# Important corrections before submission

- Do not write that the system uses “simple rules” if the project is presented as machine-learning based.
Use “trained machine learning model” instead.
- Do not list machine learning algorithms that were not actually implemented. If only Random Forest and Logistic Regression were used, remove Decision Tree and SVM from the flowchart.
- The fraud branch must not end with “Allow”. It should lead to Block/Reject and Alert.
- User Login should appear consistently in the features, application description, architecture, workflow, and flowchart.

# Chapter five

# CHAPTER FIVE
# TESTING AND RESULTS

## 5.1 Introduction

This chapter describes the testing and evaluation of the Intelligent Banking Fraud Detection System. The practical work consists of a Python machine-learning script that prepares transaction data, trains and compares three classification algorithms, and saves the selected model and preprocessing objects. A Flask web application loads these saved files and provides a browser-based interface for submitting transaction attributes and displaying a prediction with an estimated risk level. The application is an academic prototype; it is not connected to a live banking platform and does not process real bank transactions.

## 5.2 Dataset and Testing Method

The project uses the file credit_card_fraud_10k.csv, which contains 10,000 transaction records. The target column is is_fraud, where 0 represents a legitimate transaction and 1 represents a fraudulent transaction. The dataset contains 9,849 legitimate records (98.49%) and 151 fraudulent records (1.51%). The available attributes used by the model are transaction amount, transaction hour, merchant category, foreign-transaction indicator, location mismatch, device-trust score, transaction velocity during the previous 24 hours, and cardholder age.

The dataset is used for academic experimentation because access to real banking transaction records is restricted by privacy, confidentiality, and security requirements. Therefore, the results should be interpreted as results on the supplied dataset only and should not be generalized to real banking environments without further validation.

The current training script applies random oversampling to the full dataset before splitting it into training and testing subsets. It then uses an 80:20 split and applies StandardScaler, fitting the scaler on the training subset. Although the split is stratified, oversampling before the split can place duplicated minority-class records in both subsets. This creates a risk of data leakage and can make the reported test metrics overly optimistic. Consequently, the numerical results in the current version are preliminary and should not be presented as an unbiased estimate of performance on unseen transactions. [22, 23, 25]

## 5.3 Data Imbalance and Oversampling

The dataset is strongly imbalanced: fraudulent transactions account for approximately 1.51% of all records. A classifier that predicts every transaction as legitimate would achieve approximately 98.49% accuracy while failing to detect any fraudulent transaction. Accuracy alone is therefore insufficient for evaluating this task. [21]

| Class | Number of records | Percentage |
|---|---|---|
| Legitimate (0) | 9,849 | 98.49% |
| Fraudulent (1) | 151 | 1.51% |
| Total | 10,000 | 100% |

Random oversampling is used in the current implementation to increase the representation of the minority class by sampling existing fraudulent records with replacement until both classes have the same number of records. This method does not create new transaction patterns; it repeats existing examples. For a sound evaluation, the original data must first be divided into training and testing sets. Oversampling should then be applied only to the training set, while the test set should retain its original class distribution. [22]

## 5.4 Machine-Learning Models

Three supervised classification algorithms are implemented and compared:

| Model | Purpose in the experiment |
|---|---|
| Logistic Regression | A baseline classifier for comparing the other models against a relatively simple method. |
| Decision Tree | A tree-based classifier that learns decision rules from the input features. |
| Random Forest | An ensemble of decision trees that combines predictions from multiple trees. |

The script calculates Accuracy, Precision, Recall, F1-score, ROC-AUC, and a confusion matrix for each model. Precision indicates how many transactions flagged as fraud are actually fraudulent. Recall indicates how many of all fraudulent transactions are detected. F1-score balances Precision and Recall, while ROC-AUC measures discrimination across classification thresholds. The confusion matrix records true positives, true negatives, false positives, and false negatives. [24]

## 5.5 Results Reported by the Current Implementation

The existing report and saved experimental outputs identify Random Forest as the best-performing model in the current run. The reported results are summarized below for transparency. Because oversampling was performed before the train-test split, these values must be treated as preliminary implementation results rather than a verified estimate of performance on independent unseen data.

| Model | Precision | Recall | F1-score | Accuracy | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 96.03 % | 99.49 % | 97.73 % | 97.69 % | 0.9920 |
| Decision Tree | 99.70 % | 100.00 % | 99.85 % | 99.85 % | 0.9999 |
| Random Forest | 100.00 % | 100.00 % | 100.00 % | 100.00 % | 1.0000 |

In the current report, Random Forest is selected because it has the highest reported ROC-AUC and the strongest reported classification metrics. However, the perfect scores may be influenced by the data-leakage risk described in Section 5.2. The model should therefore be retrained and reevaluated after correcting the order of splitting and oversampling before the scores are treated as final.

## 5.6 Flask Web Prototype

The web interface is implemented with Flask. At startup, app.py loads the saved fraud model, feature scaler, label encoder, and feature-name list. The user enters transaction attributes through the browser interface. The backend converts the provided values into the expected feature order, encodes the merchant category, applies the saved scaler, and requests a prediction from the model. It also obtains the model's estimated probability for the fraud class and maps that value to a display label: low risk for probabilities below 0.30, medium risk for probabilities from 0.30 to below 0.60, and high risk for probabilities of 0.60 or above. [26]

These risk labels are presentation thresholds used by the prototype; they have not been established through a banking risk policy or validated for operational use. The interface demonstrates the end-to-end connection between a saved machine-learning model and a web application. It should not be described as a deployed real-time banking service.

## 5.7 Limitations

- The dataset contains 10,000 records and is not confirmed to be a real bank-provided dataset; findings are limited to this dataset.
- Oversampling is applied before the train-test split in the current script, creating a potential data-leakage problem.
- Random oversampling duplicates existing minority-class records and may increase overfitting.
- The reported perfect Random Forest scores require verification using a clean hold-out test set that was not used during resampling or training.
- The Flask application is a prototype and has not been integrated with a bank, payment gateway, or production transaction stream.
- Risk thresholds and model probabilities have not been calibrated or validated for decisions involving real customer accounts.

## 5.8 Recommendations for Further Validation

- Split the original dataset into training and testing sets before applying oversampling.
- Apply oversampling only to the training data; retain the original distribution in the test data.
- Fit all preprocessing transformations using training data only, then apply the fitted transformations to the test data. [25]
- Retrain the three classifiers and regenerate all metrics, confusion matrices, comparison charts, and saved model files.
- Report the number of test records and class distribution accurately, and ensure that all tables and figures use the same run.
- Evaluate Precision, Recall, F1-score, ROC-AUC, and false-negative counts, rather than relying on Accuracy alone.
- Test the Flask interface using the newly saved model and matching preprocessing files.
- Where possible, validate the approach on a suitable public benchmark dataset and document its source and limitations.

## 5.9 Chapter Summary

This chapter described the dataset, class imbalance, random oversampling, three classification algorithms, evaluation metrics, and the Flask-based web prototype. The current experimental outputs report Random Forest as the leading model, but the reported perfect scores cannot yet be considered conclusive because oversampling was performed before the data split. The system demonstrates the feasibility of connecting a machine-learning classifier to a web interface in an academic setting. Correcting the evaluation procedure and regenerating the results are necessary steps before making strong claims about the model's ability to generalize to unseen transactions.

# Chapter Six

## Conclusion and Future Work

### 6.1 Conclusion

The rapid expansion of digital banking services has increased the need for effective methods capable of identifying fraudulent transactions. Traditional rule-based systems are limited by their dependence on fixed conditions and their inability to adapt easily to changing fraud patterns. This project addressed that problem by developing a web-based intelligent prototype that applies machine learning techniques to classify banking transactions as legitimate or fraudulent.

The project followed a structured development process that included transaction data preparation, data preprocessing, feature analysis, handling class imbalance, training multiple classification algorithms, and evaluating their performance using Accuracy, Precision, Recall, F1-Score, AUC-ROC, and confusion matrix analysis. The system architecture integrated a web interface, a Flask backend, a trained machine learning model, and a storage layer for transaction and prediction records.

According to the experimental results reported in Chapter Five, Logistic Regression, Decision Tree, and Random Forest were compared after oversampling. The reported confusion matrices showed that Logistic Regression produced 81 false positives and 10 false negatives, while Decision Tree reduced false positives to 6 and eliminated false negatives. Random Forest was reported as the best-performing model, with 1,970 true positives, 1,970 true negatives, zero false positives, and zero false negatives on the balanced evaluation set.

The reported results also indicated that addressing class imbalance improved fraud detection performance. In the project results, Recall increased from 80.00% to 100.00%, while F1-Score increased from 88.89% to 100.00% after oversampling. These findings support the importance of data preparation and class-balancing techniques when building machine learning models for fraud detection.

The Random Forest model was selected because its ensemble structure combines the outputs of multiple decision trees and can represent nonlinear relationships between transaction features. The project further identified device trust score, transaction frequency during the previous 24 hours, and

location mismatch as important fraud indicators. The Flask-based prototype demonstrated how a trained model can be connected to a user-facing web application to produce immediate predictions.

Overall, the project achieved its main objective of designing and evaluating a machine-learning-based prototype for banking fraud detection. The developed system demonstrates the practical value of combining data preprocessing, imbalance handling, ensemble learning, and web technologies in a single academic prototype. However, the prototype should not be considered a complete production banking system, and the reported perfect testing performance requires additional validation on independent, unseen, and real-world datasets before any operational deployment.

## 6.2 Achievement of Research Objectives

The project addressed the research objectives defined in Chapter One as follows:

• Banking transaction data were prepared and processed for model development.

• Relevant fraud-related transaction features were analyzed and used as model inputs.

• The class imbalance problem was addressed using an oversampling approach during model training.

• Multiple machine learning algorithms were compared, including Logistic Regression, Decision Tree, and Random Forest.

• Model performance was evaluated using Accuracy, Precision, Recall, F1-Score, AUC-ROC, and confusion matrix values.

• Random Forest was selected as the best-performing model according to the results reported in Chapter Five.

• A Flask-based web prototype was used to demonstrate transaction input and immediate prediction.

## 6.3 Limitations of the Current Study

Although the project reported strong results, several limitations must be acknowledged to ensure that the conclusions remain academically realistic.

• The project is a web-based prototype based on historical transaction records rather than a system connected to live banking infrastructure.

• The reported perfect performance of Random Forest was obtained on the project evaluation data and may not generalize to other datasets or newly emerging fraud patterns.

• The project documentation contains an inconsistency in the imbalance-handling method: earlier chapters refer to SMOTE, while Chapter Five describes Random Oversampling.

• The evaluation section refers to both an original held-out testing set and a balanced set of 3,940 transactions; the exact evaluation procedure should therefore be documented clearly in the final report.

• Oversampling before evaluation can produce overly optimistic results if duplicated or synthetic samples influence the testing data. Balancing should be restricted to the training set.

• The available project material does not provide detailed information about the final database schema, deployment security controls, or privacy mechanisms.

• Fraud behavior changes over time, and a model trained on static historical data may lose effectiveness when transaction patterns evolve.

## 6.4 Future Work

Future development should focus on validating the prototype under more realistic conditions and extending its technical capabilities. The following improvements are recommended:

• Evaluate the selected model on a completely independent and naturally imbalanced test dataset that is not affected by oversampling.

• Resolve the methodological inconsistency by selecting and documenting one imbalance-handling method, such as Random Oversampling or SMOTE, and applying it only to the training set.

• Use cross-validation and repeated experiments to verify that the reported model performance is stable and not caused by a favorable data split.

• Test the system using larger, more diverse, and time-ordered banking datasets to measure performance against changing fraud patterns.

• Compare Random Forest with additional algorithms such as XGBoost, LightGBM, CatBoost, Support Vector Machine, anomaly detection models, and deep learning methods.

• Apply Explainable Artificial Intelligence techniques, such as feature importance analysis or SHAP, to provide understandable reasons for each fraud prediction.

• Introduce probability-based risk levels so that transactions can be categorized as low risk, suspicious, or high risk instead of relying only on a binary output.

• Develop secure customer verification and alert mechanisms for suspicious transactions.

• Add model monitoring and periodic retraining procedures to detect performance degradation and adapt to new fraud behavior.

• Strengthen data privacy and security through encryption, access control, secure logging, and privacy-preserving machine learning methods.

• Integrate the prototype with a secure database and document the complete database schema, transaction history, and audit logs.

• Deploy and test the system in a controlled cloud or banking simulation environment to evaluate latency, scalability, and reliability.

## RESEARCH QUESTIONS AND ANSWERS

**RQ1. Which machine learning algorithm provides the best performance for fraud detection?**

According to the experimental results reported in this project, Random Forest achieved the best overall performance among the evaluated models. It produced the highest reported Accuracy, Precision, Recall, and F1-Score and generated no false alarms in the presented confusion matrix. However, the reported perfect performance should be validated on an independent, naturally imbalanced dataset before real-world deployment.

**RQ2. How does data imbalance affect model performance, and how can it be handled?**

Data imbalance causes the model to favor the majority class, which consists mainly of legitimate transactions. As a result, a model may obtain high Accuracy while failing to detect fraud. This project addresses the problem using an oversampling technique. The final methodology must clearly state whether Random Oversampling or SMOTE was used. The selected balancing technique should be applied only to the training set to prevent data leakage and overly optimistic evaluation results.

**RQ3. Which features are most important for detecting fraudulent transactions?**

The project identifies transaction behavior and contextual attributes as important indicators. The most influential reported features include device trust score, transaction frequency during the previous 24 hours, and location mismatch. Transaction amount, time, type, and geographical location also contribute to identifying unusual activity.

**RQ4. How does data preprocessing improve model results?**

Data preprocessing improves the quality and consistency of the model inputs. It includes handling missing values, removing duplicate records, converting categorical variables into numerical form, scaling numerical variables when required, and maintaining the same feature order during training and prediction. These steps reduce noise and ensure that the machine learning model receives reliable input.

**RQ5. Which evaluation metrics are most suitable for banking fraud detection?**

Accuracy should not be used alone because fraud datasets are highly imbalanced. The most suitable evaluation measures are Precision, Recall, F1-Score, ROC-AUC, and the confusion matrix. Recall is especially

important because it measures how many fraudulent transactions are detected, while Precision shows how many fraud alerts are correct. F1-Score balances both measures.

## References


**[1]** Njoku, D. O., Iwuchukwu, V. C., Jibiri, J. E., Ikwuazom, C. T., Ofoegbu, C. I., & Nwokoma, F. O. (2024). Machine Learning Approach for Fraud Detection System in Financial Institution: A Web-Based Application. *International Journal of Engineering Research and Development*, 20(4), 01–12. Available at: http://www.ijerd.com.

**[2]** Detthamrong, U., Chansanam, W., Boongoen, T., & Iam-On, N. (2024). Enhancing Fraud Detection in Banking using Advanced Machine Learning Techniques. *International Journal of Economics and Financial Issues*, 14(5), 177–184. https://doi.org/10.32479/ijefi.16613

**[3]** Oduro, D. A., Okolo, J. N., Bello, A. D., Ajibade, A. T., Fatomi, A. M., Oyekola, T. S., & Owoo-Adebayo, S. F. (2025). AI-powered fraud detection in digital banking: Enhancing security through machine learning. *International Journal of Science and Research Archive*, 14(3), 1412–1420. https://doi.org/10.30574/ijsra.2025.14.3.0854

**[4]** Author(s) Unknown. (2023). Machine Learning-Based Approach for Fraud Detection. *IEEE International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE)*. https://doi.org/10.1109/IITCEE57236.2023.10091067

**[5]** Rajasekar, A., Patra, S., Dua, R., Rajmane, A., Atulker, P. K., & Sangeeta. (n.d.). Fraud Detection in Digital Banking using Machine Learning and Ensemble Models. https://doi.org/10.29284/9pnb908

**[6]** Adewumi, A. O., & Akinyelu, A. A. (2017). A survey of machine-learning and nature-inspired based credit card fraud detection techniques. *International Journal of System Assurance Engineering and Management*, 8(2), 937–953.

**[7]** Bolton, R. J., & Hand, D. J. (2002). Statistical fraud detection: A review. *Statistical Science*, 17(3), 235-255.

**[8]** Davis, J., & Goadrich, M. (2006). The relationship between Precision-Recall and ROC curves. *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 233-240.

**[9]** Chianumba, C. N., Ekwe, M. C., & Okoli, J. C. (2024). Machine learning-based fraud detection in financial transactions: A comparative analysis. *Journal of Financial Crime*, 31(2), 450–467.

**[10]** Hashemi, S. K., Mirtaheri, S. L., & Greco, S. (2022). Fraud detection in banking data by machine learning techniques. *IEEE Access*, 11, 3034–3043.

**[11]** De la Bourdonnaye, F., & Daniel, F. (2022). Evaluating resampling methods on a real-life highly imbalanced online credit card payments dataset. *arXiv preprint*.

**[12]** Chennuri, S. (2024). Advancing fraud detection in banking: Integration of data pipelines, machine learning, and cloud computing. *International Journal for Multidisciplinary Research*, 6(6).

**[13]** Zhang, R., Cheng, Y., Wang, L., Sang, N., & Xu, J. (2023). Efficient bank fraud detection with machine learning. *Journal of Computational Methods in Engineering Applications*, 3(1).

**[14]** Garcia, R., Martinez, M., & Rodriguez, J. (2019). Behaviour-based fraud detection using machine learning algorithms. *Journal of Information Security and Applications*, 46, 102-114.

**[15]** Lucas, Y., & Jurgovsky, J. (2020). Credit card fraud detection using machine learning: A survey. *arXiv preprint*.

**[16]** Liu, X., Zidek, J. V., & Sun, W. (2020). Feature engineering in fraud analytics. *Stat*, 9(1), e262.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. *IEEE Symposium Series on Computational Intelligence (SSCI)*, 159–166.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.

Hand, D. J. (2009). Measuring classifier performance: A coherent alternative to the area under the ROC curve. *Machine Learning*, 77(1), 103–123.

[17] Njoku, D. O., Iwuchukwu, V. C., Jibiri, J. E., Ikwuazom, C. T., Ofoegbu, C. I., & Nwokoma, F. O. (2024). Machine Learning Approach for Fraud Detection System in Financial Institution: A Web-Based Application. International Journal of Engineering Research and Development, 20(4), 1-12.

[18] Detthamrong, U., Chansanam, W., Boongoen, T., & Iam-On, N. (2024). Enhancing Fraud Detection in Banking using Advanced Machine Learning Techniques. International Journal of Economics and Financial Issues, 14(5), 177-184. https://doi.org/10.32479/ijefi.16613

[19] Oduro, D. A., Okolo, J. N., Bello, A. D., Ajibade, A. T., Fatomi, A. M., Oyekola, T. S., & Owoo-Adebayo, S. F. (2025). AI-powered fraud detection in digital banking: Enhancing security through machine learning. International Journal of Science and Research Archive, 14(3), 1412-1420. https://doi.org/10.30574/ijsra.2025.14.3.0854

[20] Author(s) Unknown. (2023). Machine Learning-Based Approach for Fraud Detection. IEEE International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics. https://doi.org/10.1109/IITCEE57236.2023.10091067

[21] Rajasekar, A., Patra, S., Dua, R., Rajmane, A., Atulker, P. K., & Sangeeta. (n.d.). Fraud Detection in Digital Banking using Machine Learning and Ensemble Models. https://doi.org/10.29284/9pnb908

[21] H. He and E. A. Garcia, “Learning from Imbalanced Data,” IEEE Transactions on Knowledge and Data Engineering, vol. 21, no. 9, pp. 1263–1284, 2009. doi: 10.1109/TKDE.2008.239.

[22] Scikit-learn Developers, “sklearn.utils.resample — Resample,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.utils.resample.html (accessed October 11, 2026).

[23] Scikit-learn Developers, “sklearn.model_selection.train_test_split,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html (accessed October 11, 2026).

[24] Scikit-learn Developers, “Model evaluation: quantifying the quality of predictions,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/model_evaluation.html (accessed October 11, 2026).

[25] Scikit-learn Developers, “StandardScaler,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html (accessed October 11, 2026).

[26] Pallets Projects, “Flask Quickstart,” Flask Documentation. Available: https://flask.palletsprojects.com/en/stable/quickstart/ (accessed October 11, 2026).